Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2025**

Student: Riccardo Polo Discussed with:

Solution for Project 3

Due date: Monday 07 April 2025, 18:00 CEST

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts). If you modified any provided build files, ensure they still work as expected.
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Task: Implementing the linear algebra functions and the stencil operators [50 Points]

1.1. Numerical method

We consider Fisher's reaction–diffusion equation on the square domain $\Omega = [0,1]^2$, where the population concentration $s(x,y,t)$ evolves under the combined effects of diffusion and logistic growth (eq. 1).

$$\frac{\partial s}{\partial t} = D\left(\frac{\partial^2 s}{\partial x^2} + \frac{\partial^2 s}{\partial y^2}\right)s + Rs(1-s) \tag{1}$$

The problem is complemented by constant Dirichlet boundary conditions $s = 0.1$ on $\partial\Omega$ and by an initial condition consisting of a localized perturbation, with $s = 0.2$ inside a small disk centered at $(1/4, 1/4)$ and $s = 0.1$ elsewhere.

The spatial domain is discretized on a uniform Cartesian grid with $(n+2)\times(n+2)$ points, where only the n^2 interior values are treated as unknowns. Spatial derivatives are approximated by a second-order central finite-difference scheme, yielding the standard five-point stencil, while time integration is performed using the implicit Euler method. At each time step, this results in a nonlinear algebraic system for the updated concentration field s^k.

The nonlinear system is solved with Newton's method applied to the residual equation $f(s^k) = 0$. At each Newton iteration, the corresponding linearized correction problem involving the Jacobian of the residual is solved approximately with a matrix-free Conjugate Gradient method, thereby avoiding explicit Jacobian inversion. The converged solution at time step k is then used as the initial guess for the subsequent time step.

We were asked to implement the linear algebra functions and the stencil operators, which are used in the Conjugate Gradient method and in the Newton's method, respectively.
One can look at the linear algebra functions in the file `linalg.cpp` and the stencil operators in the file `operators.cpp`.
In particular for the stencil operators we have to implement the residual function $f^k_{(i,j)}$ for the interior points, and for the points on the boundaries.
The residual function in the interior points is given by eq. 7 of the project handout. For the boundery points, we have to take into account the Dirichlet boundary conditions, obtaining:

$$\begin{aligned}
\text{East:} \quad & f^k_{(n+1,j)} = -(4+\alpha)s^k_{(n+1,j)} + s^k_{(n,j)} + 0.1 + s^k_{(n+1,j-1)} + s^k_{(n+1,j+1)} + \beta s^k_{(n+1,j)}(1 - s^k_{(n+1,j)}) + \alpha s^{k-1}_{(n+1,j)} \\
\text{West:} \quad & f^k_{(0,j)} = -(4+\alpha)s^k_{(0,j)} + 0.1 + s^k_{(1,j)} + s^k_{(0,j-1)} + s^k_{(0,j+1)} + \beta s^k_{(0,j)}(1 - s^k_{(0,j)}) + \alpha s^{k-1}_{(0,j)} \\
\text{North:} \quad & f^k_{(i,n+1)} = -(4+\alpha)s^k_{(i,n+1)} + s^k_{(i-1,n+1)} + s^k_{(i+1,n+1)} + 0.1 + s^k_{(i,n)} + \beta s^k_{(i,n+1)}(1 - s^k_{(i,n+1)}) + \alpha s^{k-1}_{(i,n+1)} \\
\text{South:} \quad & f^k_{(i,0)} = -(4+\alpha)s^k_{(i,0)} + s^k_{(i,1)} + 0.1 + s^k_{(i-1,0)} + s^k_{(i+1,0)} + \beta s^k_{(i,0)}(1 - s^k_{(i,0)}) + \alpha s^{k-1}_{(i,0)} \\
\text{NE corner:} \quad & f^k_{(n+1,n+1)} = -(4+\alpha)s^k_{(n+1,n+1)} + s^k_{(n,n+1)} + 0.1 + s^k_{(n+1,n)} + \beta s^k_{(n+1,n+1)}(1 - s^k_{(n+1,n+1)}) + \alpha s^{k-1}_{(n+1,n+1)} \\
\text{NW corner:} \quad & f^k_{(0,n+1)} = -(4+\alpha)s^k_{(0,n+1)} + 0.1 + s^k_{(1,n+1)} + s^k_{(0,n)} + \beta s^k_{(0,n+1)}(1 - s^k_{(0,n+1)}) + \alpha s^{k-1}_{(0,n+1)} \\
\text{SE corner:} \quad & f^k_{(n+1,0)} = -(4+\alpha)s^k_{(n+1,0)} + s^k_{(n,0)} + 0.1 + s^k_{(n+1,1)} + \beta s^k_{(n+1,0)}(1 - s^k_{(n+1,0)}) + \alpha s^{k-1}_{(n+1,0)} \\
\text{SW corner:} \quad & f^k_{(0,0)} = -(4+\alpha)s^k_{(0,0)} + 0.1 + s^k_{(1,0)} + s^k_{(0,1)} + \beta s^k_{(0,0)}(1 - s^k_{(0,0)}) + \alpha s^{k-1}_{(0,0)}
\end{aligned}$$

The `main.cpp` implements the time integration of the nonlinear system arising from the implicit discretization of Fisher's equation.
For each time step k, Newton's method is applied to solve the nonlinear system $f(s^k) = 0$. At iteration m, the residual is computed as:

$$r^{(m)} = f\big(s^{k,m}\big),$$

which corresponds to the call to the stencil operator `diffusion`. The convergence of the nonlinear iteration is monitored through the Euclidean norm of the residual:

$$\|r^{(m)}\|_2.$$

If the residual falls below a prescribed tolerance, the Newton iteration is terminated.
Otherwise, the Newton correction $\delta s^{(m)}$ is obtained by approximately solving the linearized system

$$J\big(s^{k,m}\big)\,\delta s^{(m)} = r^{(m)},$$

where J denotes the Jacobian of the residual. This system is solved using a matrix-free Conjugate Gradient method, implemented in `hpc_cg`, which avoids the explicit construction of the Jacobian matrix.

If the CG solver converges, the solution is updated according to the Newton step:

$$s^{k,m+1} = s^{k,m} - \delta s^{(m)},$$

implemented via the BLAS-like operation `hpc_axpy`.

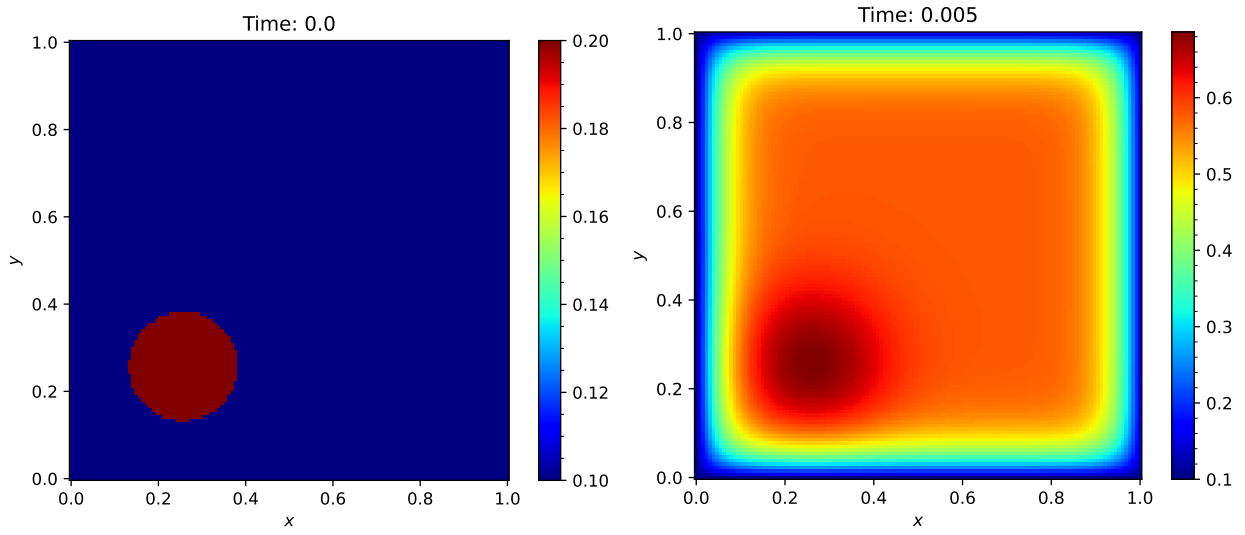
The process is repeated until either convergence is reached or a maximum number of Newton iterations is exceeded. If convergence is not achieved, the simulation is terminated and an error message is reported.

1.2. Results

The implementation of the linear algebra functions and the stencil operators was successful, as the program runs without errors and produces results consistent with the expected behavior of the solution to Fisher's equation.
In particular the output is given by the following listing

```
================================================================================
                    Welcome to mini-stencil!
version :: C++ Serial
mesh :: 128 * 128 dx = 0.00787402
time :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
simulation took 0.157575 seconds
1513 conjugate gradient iterations, at rate of 9601.76 iters/second
300 Newton iterations
--------------------------------------------------------------------------------
### 1, 128, 100, 1513, 300, 0.157575
```

Looking at the project handout and comparing with our results, we can see that the number of CG iterations and Newton iterations are consistent with the expected values, which indicates that our implementation is correct.
We can also see in figures 1a and 1b the initial and final solution of the problem, which are consistent with the images provided in the project handout, confirming the correctness of our implementation.



(a) Initial solution

(b) Final solution

Figure 1: Comparison between initial and final solution

2. Task: Adding OpenMP to the nonlinear PDE mini-app [50 Points]

2.1. Welcome message in `main.cpp` and serial compilation [5 points]

To solve this task we have modified the `main.cpp` file with the modifications shown in listing 1.

Listing 1: Modifications to the welcome message in `main.cpp` to support OpenMP

```
[...]

#ifdef _OPENMP
#include <omp.h>
#endif

[...]

#ifdef _OPENMP
int threads = omp_get_max_threads();
#endif
#ifndef _OPENMP
int threads = 1;
#endif

[...]

#ifdef _OPENMP
std::cout << "version :: C++ OpenMP" << std::endl;
#endif
#ifndef _OPENMP
std::cout << "version :: C++ serial" << std::endl;
#endif

[...]
```

2.2. Linear algebra kernel [10 Points]

All the functions in the file `linalg.cpp` were parallelized using the `#pragma omp parallel for` directive to parallelize the loops.
For the function `hpc_norm2` we have to take into account the reduction of the sum, which is done using the `reduction(+:sum)` clause.

2.3. The diffusion stencil [15 Points]

The parallelization of the `diffusion()` routine is based on OpenMP loop-level parallelism.
The largest amount of work is associated with the interior points, since they constitute almost the whole domain. For this reason, the interior stencil evaluation is parallelized with

```
#pragma omp parallel
{
    #pragma omp for collapse(2) nowait
    for (int j = 1; j < jend; j++) {
        for (int i = 1; i < iend; i++) {
            ...
        }
    }
    ...
}
```

The `#pragma omp parallel` directive creates a team of threads only once, and all subsequent loop worksharing constructs are executed inside the same parallel region. This is more efficient than creating a new parallel region for every loop separately.

For the interior update, the directive `#pragma omp for collapse(2)` is used. The `collapse(2)` clause merges the two nested loops over i and j into a single iteration space.

After the interior points, the four non-corner boundaries are treated in separate loops:

- east boundary,
- west boundary,
- north boundary,
- south boundary.

These loops are also parallelized with `#pragma omp for`. They are kept separate because the stencil on each boundary differs from the interior stencil due to the Dirichlet boundary values `bndE`, `bndW`, `bndN`, and `bndS`.

For the first boundary loops, the `nowait` clause is added:

```
#pragma omp for nowait
```

This removes the implicit barrier at the end of each loop. Since the different boundary updates are independent, there is no need for all threads to synchronize after every single boundary loop. Avoiding these unnecessary barriers reduces synchronization overhead and improves performance.

The four corner points are computed outside the parallel region and therefore serially. This choice is reasonable because only four grid points are involved, so the computational cost is negligible compared to the interior and edge updates. Parallelizing these few operations would introduce extra overhead without any meaningful speedup.

An important property of this implementation is that it is free of data races. Each iteration writes to a distinct entry of the residual field $f(i, j)$, while the fields `s_old`, `s_new`, and the boundary arrays are only read. Therefore, no two threads update the same memory location simultaneously.

2.4. Strong scaling [10 points]

In the strong scaling test we have measured the time to solution for different combinations of number of threads and problem sizes.
In particular, we have measured the time to solution for NCPU = 1, 2, 4, 8, 16 threads across resolutions of $n \times n$, where n = 64, 128, 256, 512, 1024.
The results are shown in figure 2, where we can see the strong scaling speedup of the OpenMP implementation compared to the serial version.

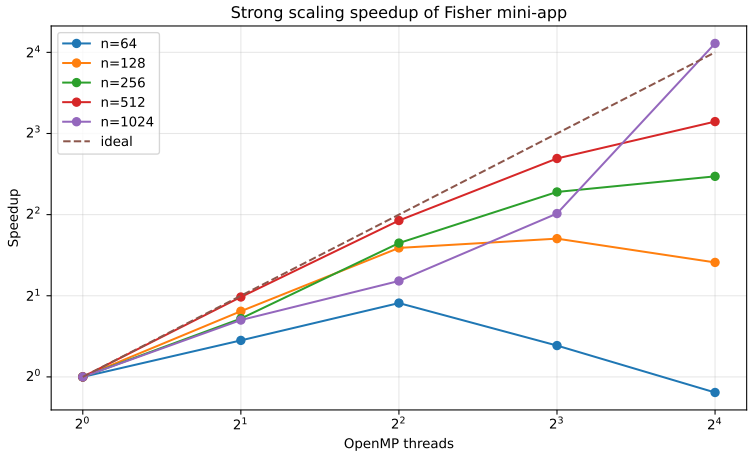


Figure 2: Strong scaling speedup of the OpenMP implementation compared to the serial version.

2.5. Weak scaling [10 points]

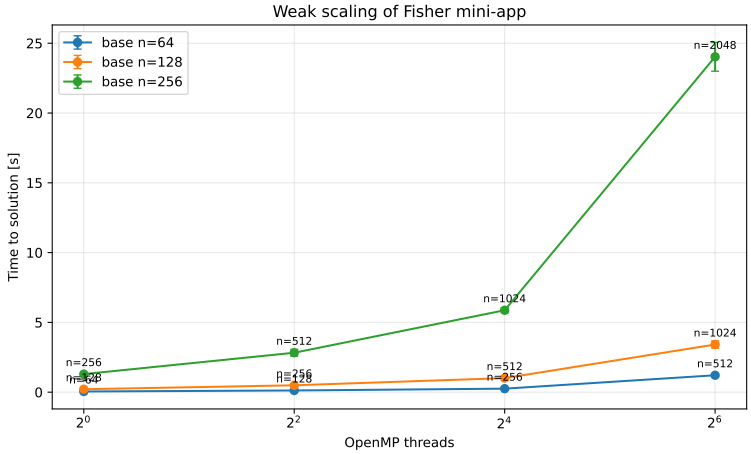


Figure 3: Weak scaling time vs. number of threads for the OpenMP implementation.